

Die Spiele-Schmiede: Vom Anfänger zum Spiele-Programmierer (Zero to Hero)

Herzlich willkommen in deiner eigenen Spiele-Werkstatt! In diesem Handbuch lernst du Schritt für Schritt, wie du mit echtem C++ eigene Retro-Spiele (wie Pong, Snake, Breakout oder Space-Shooter) programmierst.

Kapitel 0: Das große Geheimnis der Videospiele

Hast du schon mal ein **Daumenkino** gebastelt? Wenn man schnell durch die Seiten blättert, erwachen die gezeichneten Figuren zum Leben!

Genauso funktioniert dein Computer:

- Unser Spiel malt **60-mal in jeder einzelnen Sekunde** ein neues Bild auf den Bildschirm.
- Das Herzstück jedes Spiels ist diese eine Funktion:

```
cpp

void update(Engine& e) {

    // Alles hier drin wird 60-mal pro Sekunde ausgeführt!

}
```



Das C++ Zauberbuch: Die wichtigsten Grundlagen

C++ ist eine der schnellsten und mächtigsten Programmiersprachen der Welt. Fast alle großen Konsolen- und PC-Spiele werden in C++ geschrieben. Hier sind die 6 wichtigsten Bausteine, die du brauchst:

1. Variablen (Die beschrifteten Schachteln)

In Variablen merkt sich der Computer Zahlen, Texte und Zustände:

- `int` (Ganze Zahlen): Für Punktestände, Leben oder Zähler.

```
cpp

int score = 0;

int leben = 3;
```

`

- **float** (Kommazahlen): Für Positionen und Geschwindigkeiten (damit Bewegungen butterweich sind).

`

cpp

```
float ballX = 160.0f;
```

```
float geschwindigkeit = 3.5f;
```

`

- **bool** (Schalter: **true** / **false**): Für Ja/Nein-Zustände.

`

cpp

```
bool spielAktiv = true;
```

```
bool laserBereit = false;
```

`

2. Entscheidungen treffen (**if** und **else**)

Mit **if** („Wenn...“) kann dein Spiel auf Ereignisse reagieren:

```
if (ballX > 320) {
    score = score + 1;      // Tor erzielt!
    e.play_tone(Notes::C5);
} else {
    // Sonst: Ball ist noch im Feld
}
```

Die Vergleichs-Zeichen:

- **==** Ist gleich? (**if (leben == 0)**)
- **!=** Ist ungleich? (**if (richtung != Direction::Down)**)
- **<**, **>**, **<=**, **>=** Kleiner, Größer, Kleiner-Gleich, Größer-Gleich
- **&&** **UND** (**if (ballX > 0 && ballX < 320)**)
- **||** **ODER** (**if (e.key(Key::W) || e.key(Key::Up))**)

3. Wiederholungen (**for** -Schleifen)

Wenn du viele Dinge auf einmal tun willst (z. B. 70 Sterne zeichnen oder 5 Äpfel platzieren), benutzt du eine **for** -Schleife:

```
// Zählt von 0 bis 4 (wiederholt sich genau 5-mal):
for (int i = 0; i < 5; ++i) {
    spawnApple();
}
```

4. Eigene Funktionen & Methoden (Eigene Zaubersprüche)

Statt denselben Code 10-mal zu schreiben, packen wir ihn in eine Funktion mit eigenem Namen:

```
// Eigener Zauberspruch zum Abfeuern:
void feuerLaser(Engine& e) {
    e.play_tone(Notes::As5, 0.04f);
    lasers.push_back({playerX + 7, playerY, -300.0f, true});
}
```

5. Die `std` Standard-Bibliothek (Schlaue Werkzeuge)

C++ bringt einen Werkzeugkasten namens `std::` (Standard Library) mit:

- `std::vector` (Die flexible Liste):

Ein Vector ist wie ein Gummiband-Array. Er wächst automatisch, wenn neue Dinge dazukommen:

```
` cpp
```

```
std::vector stars; // Liste aller Sterne
```

```
stars.push_back(neuerStern); // Stern hinten anfügen
```

```
stars.pop_back(); // Letzten Stern entfernen
```

```
int anzahl = stars.size(); // Wie viele Sterne haben wir?
```

```
`
```

- `std::deque` (Die Schlange / Kette):

Perfekt für Snake! Bei einer Deque („Double Ended Queue“) kannst du blitzschnell vorne anbauen und hinten abschneiden:

```
` cpp
```

```
body.push_front(neuerKopf); // Vorne neuen Kopf anfügen
```

```
body.pop_back(); // Hinten Schwanzteil abschneiden
```



- **std::clamp** (Der Begrenzer):

Hält einen Wert sauber zwischen einem Minimum und Maximum (damit Schläger nicht aus dem Bildschirm fahren):

```
cpp
```

```
playerY = std::clamp(playerY, 18.0f, 200.0f);
```



6. Echtes Hacken: Dein Spiel im Terminal kompilieren!

Ein **Compiler** übersetzt deinen C++ Code in Maschinensprache, die der Prozessor direkt ausführt.

Du kannst dein Spiel direkt im Terminal (Shell) von Hand kompilieren:

```
# 1. Den C++ Compiler 'g++' aufrufen:
g++ -std=c++17 -I/opt/homebrew/include main.cc -o mein_spiel -L/opt/homebrew/lib

# 2. Dein fertiges Spiel starten:
./mein_spiel
```

Oder noch einfacher mit unserem **Makefile**:

```
make          # Kompiliert das Spiel
./test        # Startet das Spiel
make static   # Baut ein eigenständiges Programm (ohne externe Abhängigkeiten)
make bundle   # Baut eine echte macOS App: MTG.app!
```

Level 1: Die Malerwerkstatt (Farben & Formen)

1. Die Zauberleinwand & Koordinaten

Unser Bildschirm ist **320 Pixel breit** und **240 Pixel hoch**.

- **X (nach rechts)**: Geht von 0 (ganz links) bis 320 (ganz rechts).
- **Y (nach unten)**: Geht von 0 (ganz oben) bis 240 (ganz unten).

- Die Ecke oben links ist **(0, 0)**!

2. Der Farbkasten

Du kannst diese Farben überall verwenden:

- `Colors::DarkGreen` (Dunkelgrüner Rasen)
- `Colors::Red` (Leuchtendes Rot)
- `Colors::Blue` (Helles Blau)
- `Colors::Yellow` (Sonnengelb)
- `Colors::White` (Schneeweiß)
- `Colors::Black` (Tiefschwarz)
- `Colors::Orange` , `Colors::LightGreen` , `Colors::DarkGray` ,
`Colors::LightGray`

3. Deine Mal-Werkzeuge

Befehl	Was er macht	Beispiel
<code>e.cls(Farbe)</code>	Tafel putzen: Malt den ganzen Bildschirm mit einer Farbe an.	<code>e.cls(Colors::DarkGreen);</code>
<code>e.pset(x, y, Farbe)</code>	Punkt malen: Setzt einen einzelnen Pixel.	<code>e.pset(160, 120, Colors::Yellow);</code>
<code>e.line(x1, y1, x2, y2, Farbe)</code>	Linie ziehen: Verbindet zwei Punkte mit einem Strich.	<code>e.line(0, 0, 320, 240, Colors::White);</code>
<code>e.rect(x, y, b, h, Farbe)</code>	Rechteck-Rahmen: Zeichnet den Umriss eines Kastens.	<code>e.rect(10, 10, 50, 30, Colors::Red);</code>
<code>e.rectfill(x, y, b, h, Farbe)</code>	Rechteck füllen: Zeichnet einen ausgefüllten Kasten.	<code>e.rectfill(10, 10, 50, 30, Colors::Red);</code>
<code>e.circle(x, y, r, Farbe)</code>	Kreis-Rahmen: Zeichnet den Umriss eines Kreises.	<code>e.circle(160, 120, 20, Colors::White);</code>
<code>e.circlefill(x, y, r, Farbe)</code>	Ball malen: Zeichnet einen ausgefüllten Kreis.	<code>e.circlefill(160, 120, 8, Colors::Yellow);</code>

Befehl	Was er macht	Beispiel
<code>e.draw_text(x, y, "TEXT", Farbe)</code>	Schrift schreiben: Schreibt Text auf den Bildschirm.	<code>e.draw_text(100, 50, "HALLO WELT", Colors::White);</code>
<code>e.draw_digit(x, y, zahl, Farbe)</code>	Große Zahl: Schreibt Punktestände.	<code>e.draw_digit(100, 20, 5, Colors::Yellow, 3);</code>

🎯 **Mini-Quest 1:** Male ein Haus mit einem dreieckigen Dach (aus Linien) und einer bunten Tür!

🏃 Level 2: Leben einhauchen! (Variablen & Bewegung)

Was ist eine Variable?

Eine Variable ist wie eine **beschriftete Schachtel**, in der sich eine Zahl merkt:

```
float ballX = 100.0f; // Eine Schachtel namens ballX mit dem Wert
```

Die Zauberformel für Bewegung

Wenn wir in jedem Bild (`update`) die Zahl ein kleines Stückchen verändern, bewegt sich das Objekt:

```
class MeinSpiel : public Game {
    float ballX = 0.0f;

    void update(Engine& e) override {
        e.cls(Colors::DarkGreen);

        // Den Ball ein Stück nach rechts schieben:
        ballX = ballX + 2.0f;

        // Ball an der neuen Position malen:
        e.circlefill(ballX, 120, 8, Colors::Yellow);
    }
};
```

Level 3: Die Fernbedienung (Tastatur & Maus)

Jetzt machen wir das Spiel interaktiv!

Tasten abfragen:

- `e.key(Key::...)` → Prüft, ob eine Taste **gedrückt gehalten** wird (perfekt für Schläger / Raumschiffe):

```
cpp

if (e.key(Key::W)) {

    spielerY = spielerY - 4.0f; // Nach oben fliegen

}

if (e.key(Key::S)) {

    spielerY = spielerY + 4.0f; // Nach unten fliegen

}
```

- `e.pressed(Key::...)` → Prüft, ob eine Taste **gerade angetippt** wurde (perfekt für Schießen oder Springen):

```
cpp

if (e.pressed(Key::Space)) {

    e.play_tone(Notes::As5, 0.1f); // Schuss-Ton abspielen!
```

}

Verfügbare Tasten:

- `Key::W` , `Key::S` , `Key::A` , `Key::D`
- `Key::Up` , `Key::Down` , `Key::Left` , `Key::Right`
- `Key::Space` , `Key::Enter` , `Key::Escape`
- `Key::Num1` , `Key::Num2` , `Key::Num3`

Mit der Maus malen:

- `e.mouse_x()` und `e.mouse_y()` verraten dir die Position des Mauszeigers.
- `e.mouse_down()` prüft, ob die Maustaste gedrückt ist:

```
cpp

if (e.mouse_down()) {

    e.circlefill(e.mouse_x(), e.mouse_y(), 5, Colors::Yellow);

}
```

🔥 Level 4: Kollisionen & Spielphysik

1. Wand-Abprall (Der Flummi-Trick)

Wenn der Ball oben oder unten anstößt, kehren wir einfach seine vertikale Geschwindigkeit (`ballVy`) um:

```
ballY = ballY + ballVy;

if (ballY <= 10 || ballY >= 230) {
    ballVy = -ballVy; // Richtung umdrehen
    e.play_tone(Notes::G4, 0.05f); // Abprall-Ton!
}
```

2. Schläger trifft Ball (Kollisionsprüfung)

Wir prüfen, ob sich der Ball innerhalb der Schläger-Grenzen befindet:

```

if (ballX <= schlaegerX + schlaegerBreite &&
    ballY >= schlaegerY && ballY <= schlaegerY + schlaegerHoehe)
{
    ballVx = -ballVx; // Ball fliegt wieder
    e.play_tone(Notes::G5, 0.08f); // Treffer-Ton!
}

```

🎵 Level 5: Töne & Musik (Der Noten-Baukasten)

Töne machen ein Spiel erst richtig lebendig! Du musst dir keine komplizierten Hertz-Zahlen (wie 311.13 Hz) merken, sondern kannst einfach echte **Notennamen** (`Notes::...`) verwenden:

1. Die Notennamen (`Notes::...`)

- **Oktave 3 (Bass):** `Notes::C3` , `Notes::E3` , `Notes::G3` , `Notes::A3` , etc.
- **Oktave 4 (Klavier-Mitte):** `Notes::C4` (Mittleres C), `Notes::D4` , `Notes::E4` , `Notes::F4` , `Notes::G4` , `Notes::A4` (Kammerton 440 Hz), `Notes::B4`
- **Oktave 5 (Hohe Melodie):** `Notes::C5` , `Notes::D5` , `Notes::E5` , `Notes::F5` , `Notes::G5` , `Notes::A5` , `Notes::B5`
- **Oktave 6 (Piepser/Glocken):** `Notes::C6` , `Notes::E6` , `Notes::G6`
- *Tipp für Halbtöne:* Ein `s` steht für Kreuz (#) wie `Notes::Fs4` (Fis / F#) oder `Notes::Cs5` (Cis / C#).

2. Einzelne Töne abspielen:

`e.play_tone(Note, DauerInSekunden) :`

- **Heller Piep-Ton (z. B. Sprung / Treffer):**

```
e.play_tone(Notes::G5, 0.08f);
```

- **Tiefer Wumm (z. B. Wandstoß / Explosion):**

```
e.play_tone(Notes::C3, 0.20f);
```

- **Sehr hoher Münzen-Sound:**

```
e.play_tone(Notes::E6, 0.06f);
```

3. Eigene Melodien komponieren:

Mit `e.play_melody` kannst du mehrere Noten nacheinander abspielen:

```
// Eine fröhliche 3-Ton-Fanfare:
e.play_melody({
  {Notes::C5, 0.10f}, // Ton C
  {Notes::E5, 0.10f}, // Ton E
  {Notes::G5, 0.25f} // Ton G (länger)
});

// Eine traurige Verlierer-Melodie:
e.play_melody({
  {Notes::A4, 0.15f},
  {Notes::Fs4, 0.15f},
  {Notes::Ds4, 0.18f},
  {Notes::A3, 0.40f}
});
```

Level 6: Spiel 1 – Das Retro-Pong Match ([PongGame.hpp](#))

In **Pong** lernen wir, wie zwei Spieler gegeneinander antreten:

1. **Zwei Schläger:** Spieler 1 (Rot mit `W / S`), Spieler 2 (Blau mit Pfeiltasten).
2. **Dynamischer Abprallwinkel:** Trifft der Ball die Schlägermitte, fliegt er gradeaus. Trifft er den Rand, prallt er im steilen Winkel ab!
3. **Tempo-Steigerung:** Bei jedem Schlag wird der Ball 5% schneller (`speed *= 1.05f`).

Level 7: Spiel 2 – Das Schlangen-Duell ([SnakeGame.hpp](#))

In **Snake** lernen wir zwei mächtige neue Programmier-Tricks kennen:

1. Das Kachel-Gitter (Grid)

Statt in einzelnen Pixeln rechnen wir hier in Kacheln (jede Kachel ist 8x8 Pixel groß):

- Unser Spielfeld hat **40 Kacheln Breite** und **30 Kacheln Höhe**.
- Eine Kachel-Position ist einfach:

```
cpp
```

```
struct Cell { int x, y; };
```

```
`
```

2. Wie bewegt sich eine Schlange?

Die Schlange ist eine Kette von Kacheln (`std::deque`). In jedem Schritt passiert Magie:

```
// 1. Vorne einen neuen Kopf in Gehrictu
body.push_front(neuerKopf);

// 2. Hat die Schlange einen Apfel gefres
if (apfelGefressen) {
    score++; // Schlange wächst automatis
} else {
    body.pop_back(); // Schwanz nachziehe
}
```

3. Der „Apfel-Klau“-Mechanismus (Modell B)

Wenn eine Schlange an die Wand oder in den Gegner fährt:

```
// Jeden Körperteil der Schlange in einen
for (size_t i = 1; i < body.size(); ++i)
    apples.push_back(body[i]);
}
// Schlange startet sofort wieder klein i
respawnSnake();
```

Der überlebende Spieler kann die Trümmer-Äpfel blitzschnell fressen und riesig werden!

🚀 Level 8: Spiel 3 – Die Weltraum-Schlacht ([StarfighterGame.hpp](#))

Hier steigen wir ein in die Welt der **Partikel-Systeme** und **Pixel-Art-Schiffe**:

1. Das 3D-Parallaxe Sternenfeld

Um dem Weltraum echte Tiefe zu verleihen, lassen wir 70 Sterne in **3 verschiedenen Tiefen-Ebenen** fliegen:

- **Hintergrund** (dunkelgrau, langsam: 25 px/s) → Weit entfernte Sterne
- **Mittlere Sterne** (hellgrau, mittel: 65 px/s)
- **Vordergrund** (strahlend weiß, schnell: 130 px/s) → Nahe Sterne

```
struct Star {
    float x, y, speed;
    uint8_t color;
};

// In jedem Frame nach unten wandern:
s.y += s.speed * e.dt();
if (s.y >= 240.0f) { s.y = 0.0f; s.x = ra
```

2. 16x16 Pixel-Art Raumschiffe mit Bitmasken

Wir können Raumschiffe direkt im Code als Pixelmuster zeichnen:

```
// Das 16x16 Delta X-Wing Schiff (Zeile f
static const uint16_t playerSprite[16] =
    0b0000000110000000, // Spitze
    0b0000000111100000, // Cockpit
    0b0100001111000010, // Flügelkanonen vo
    0b0100111111110010, // Delta-Flügel
    0b1111111111111111, // Rumpf
    0b0000000100100000, // Schubflamme
};
```

3. Schiffs-Explosionen mit echter Physik (Trägheit & Farbverlauf)

Wenn ein Schiff explodiert, sprengen sich die Trümmer nicht nur im Kreis auseinander, sondern **nehmen den vollen**

Geschwindigkeitsvektor des Schiffs mit:

```
// 1. Schiffsimpuls + radiale Sprengung
p.vx = shipVx + burstSpeed * cos(angle);
p.vy = shipVy + burstSpeed * sin(angle);

// 2. Thermischer Farbverlauf über die Le
// Frisch (100%-75%): Weißer Blitz -> Gel
```

Level 9: Die Retro-Konsole (MenuGame.hpp)

Wie baut man ein Hauptmenü, um zwischen mehreren Spielen umzuschalten?

```
class MenuGame : public Game {
    std::unique_ptr<PongGame> pong;
    std::unique_ptr<SnakeGame> snake;
    std::unique_ptr<StarfighterGame> star

    void update(Engine& e) override {
        // Mit Escape immer zurück ins Me
        if (e.pressed(Key::Escape)) { cur

        if (e.pressed(Key::Num1)) { curre
        if (e.pressed(Key::Num2)) { curre
        if (e.pressed(Key::Num3)) { curre
    }
};
```



Der Spickzettel (Cheat Sheet) auf einen Blick

```
// --- Malen ---
e.cls(Farbe);
e.pset(x, y, Farbe);
e.line(x0, y0, x1, y1, Farbe);
e.rectfill(x, y, breite, hoehe, Farbe);
e.circlefill(x, y, radius, Farbe);
e.draw_text(x, y, "TEXT", Farbe);
e.draw_digit(x, y, zahl, Farbe, 3);

// --- Steuerung ---
if (e.key(Key::W)) { ... }
if (e.pressed(Key::Space)) { ... }
e.mouse_x();
e.mouse_y();
e.mouse_down();

// --- Sound & Noten ---
e.play_tone(Notes::C5, 0.1f);
e.play_melody({ {Notes::C5, 0.1f}, {Notes

// --- Farben ---
Colors::DarkGreen, Colors::Red, Colors::B
Colors::Yellow, Colors::White, Colors::Bl
Colors::Orange, Colors::LightGreen, Color
```

**Viel Spaß beim Erfinden und Programmieren
deiner eigenen Spiele! 🚀**